

Inhaltsverzeichnis

1. Datentypen	1
1.1. Primitive Datentypen	1
1.2. Typumwandlungen primitiver Datentypen	1
1.3. Komplexe Datentypen (Klassen)	3
1.4. Typumwandlungen komplexer Datentypen	4

1. Datentypen

1.1. Primitive Datentypen

Übersicht über die Datentypen in Java:

Typname	Größe ^[1]	Wrapper-Klasse	Wertebereich	Beschreibung
boolean	undefiniert ^[2]	java.lang.Boolean	true / false	Boolescher Wahrheitswert, Boolescher Typ ^[3]
char	16 bit	java.lang.Character	0 ... 65.535 (z. B. 'A')	Unicode-Zeichen (UTF-16)
byte	8 bit	java.lang.Byte	-128 ... 127	Zweierkomplement-Wert
short	16 bit	java.lang.Short	-32.768 ... 32.767	Zweierkomplement-Wert
int	32 bit	java.lang.Integer	-2.147.483.648 ... 2.147.483.647	Zweierkomplement-Wert
long	64 bit	java.lang.Long	-2 ⁶³ bis 2 ⁶³ -1, ab Java 8 auch 0 bis 2 ⁶⁴ -1 ^[4]	Zweierkomplement-Wert
float	32 bit	java.lang.Float	+/-1,4E-45 ... +/-3,4E+38	32-bit IEEE 754, es wird empfohlen, diesen Wert nicht für Programme zu verwenden, die sehr genau rechnen müssen.
double	64 bit	java.lang.Double	+/-4,9E-324 ... +/-1,7E+308	64-bit IEEE 754, doppelte Genauigkeit

Quelle: → [Wikibooks: Datatypes](#)

Quelle: → [Oracle: Datatypes](#)

1.2. Typumwandlungen primitiver Datentypen

Type-Casting mit primitiven Datentypen

Man unterscheidet zwischen einer **expliziten** und einer **impliziten** Typumwandlung.

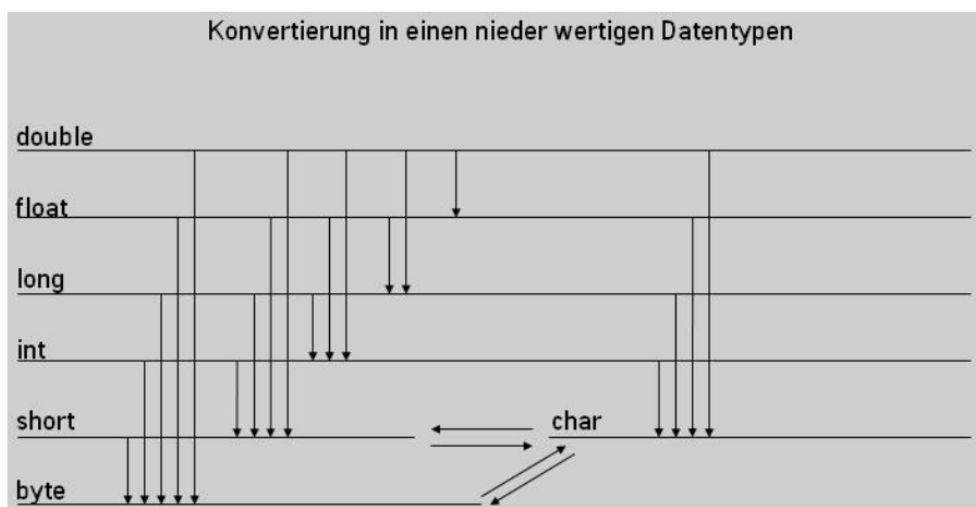
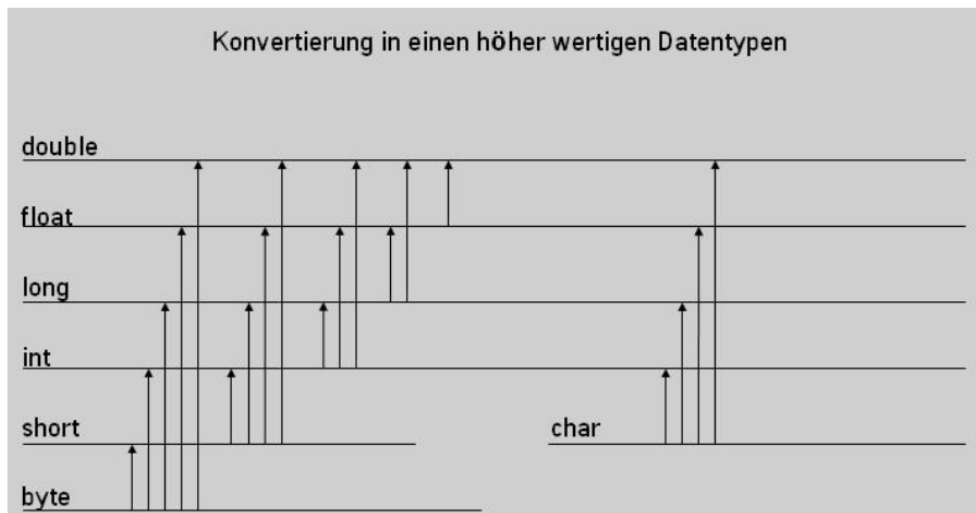
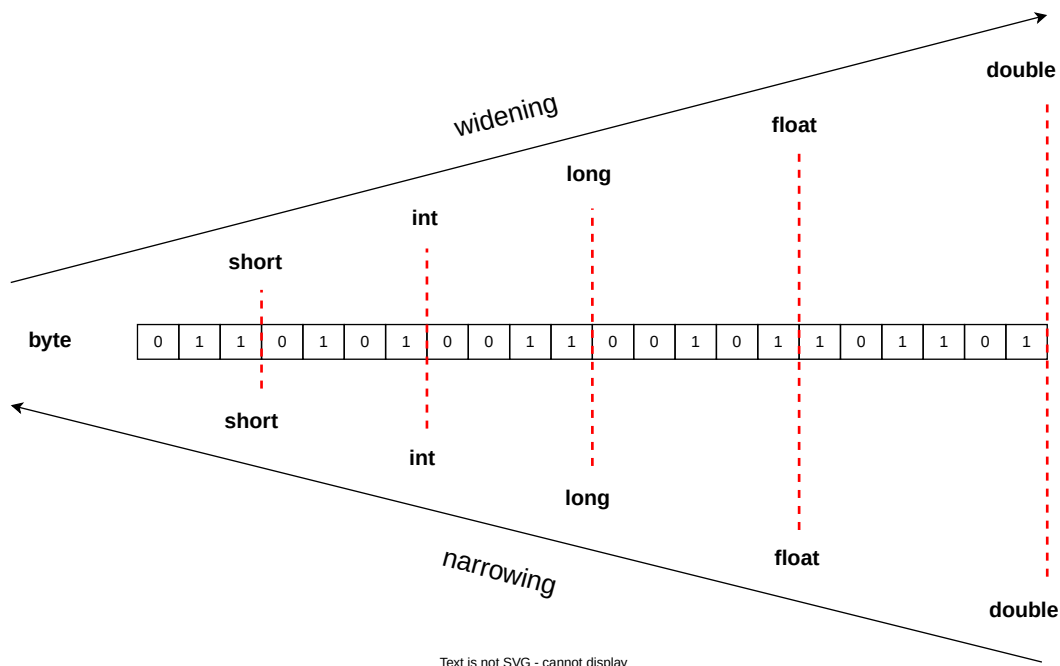


Figure 1. Widening & Narrowing



Bei der Umwandlung in "kleinere" Datentypen können Fehler auftreten (Informationsverlust), es findet das sogenannte "Narrowing" automatisch statt. Es entsteht zwar kein Compiler-Fehler, aber z.B. die Zahl wird verfälscht (→ [Narrowing](#))

Eine andere Darstellung:



Implizite Typumwandlung

Die implizite Typumwandlung findet automatisch bei der Zuweisung statt. Dies geht jedoch nur, wenn ein niederwertiger Datentyp in einen höher wertigeren Datentypen umgewandelt wird, also z.B. vom Datentyp `int` in den Datentyp `long`:

```
int wert1 = 10;
long wert2 = 30;
wert2 = wert1; // automatische Umwandlung
```

Explizite Typumwandlung (Type Casting)

Die explizite Umwandlung erfolgt durch den sogenannten `cast`-Operator mit runden Klammern. Hier wird von einem höher wertigeren Datentyp in einen nieder wertigen Datentypen umgewandelt. In welchen Datentyp umgewandelt werden soll, muss bei dem `cast` Operator explizit angegeben werden.

```
int wert1 = 10;
float wert2 = 30.5f;
wert1 = (int) wert2; // Umwandlung per 'cast'
```

1.3. Komplexe Datentypen (Klassen)

Komplexe Datentypen in Java sind Datentypen, die sich auf Objekte beziehen und in Form von Klassen implementiert werden. Im Gegensatz zu primitiven Datentypen sind sie nicht vordefiniert, sondern werden von Menschen implementiert.

Hauptarten von komplexen Datentypen:

Es gibt ...

selbst-implementierte Klassen

Sie dienen als Vorlage für Objekte und enthalten verschiedene Komponenten wie Namen, Attribute, Modifikatoren und Methoden.

und andererseits von Java mitgelieferte Klassen wie ...

Strings

Ermöglichen die Darstellung von Abfolgen von Zeichen.

Arrays & Listen

Ermöglichen das Speichern mehrerer Werte.

Interfaces

Fungieren als Abstraktionen und definieren Schnittstellen für andere Klassen.

Enums

Ermöglichen die Definition unveränderlicher Werte mit festgelegten Werten.

Wrapper (-Klassen)

Für jeden primitiven Datentyp gibt es eine entsprechende **Wrapper-Klasse**, die neben dem Wert zusätzliche Methoden anbietet. Zum Beispiel:

```
int i = 42;  
Integer j = 42; // Erzeugt ein Objekt des Wrappers 'Integer'
```

Komplexe Datentypen in Java bieten große Flexibilität bei der Strukturierung und Organisation von Daten. Sie ermöglichen die Erstellung von benutzerdefinierten Datenstrukturen und Objekten, was für die objektorientierte Programmierung von besonderer Bedeutung ist.

1.4. Typumwandlungen komplexer Datentypen



Übergang zum Modul: Komplexe Datentypen in Form von Klassen → [Klassen](#) & [Instanzen](#), dann ggf. wieder hierher zurück!

Auch **komplexe Datentypen** - also Instanzen von Java **Klassen** - können den Typen wechseln. Das ist natürlich ebenso klaren Regeln unterworfen.

So z.B. kann eine Klasse **Regionalzug** den Datentypen **Zug** bekommen, wenn die Klassen in einer Vererbungshierarchie stehen, ähnlich bei **Interfaces**. Das haben wir insb. schon im vorigen Modul **module-classes** gesehen und genutzt.

Das ist also insbesondere im Kontext der Vererbung interessant bzw. von Nutzen:

Beispiel:

```
public class HumanBeing {  
    public String name;  
}  
  
public class German extends HumanBeing {  
    public String country;  
}  
  
HumanBeing human = new German();  
human.name = "Johnny Walker"; ①  
  
German german = (German)human; ②  
german.country = "Germany";
```

- ① Zugriff auf das Feld `name` der abstrakten Klasse `Human Being`
- ② Zugriff auf das Feld `country` der konkreten Klasse `German` erst NACH dem **Down-Casting** möglich.

Demo/Beispiele:

→ `src/test/java/de/dhbw/demo/DatatypesDemoTest.java`

Übungen:

Übung 1 - Typumwandlung/Casting

→ `src/test/java/de/dhbw/exercise/DatatypesExerciseTest.java`

Schreibe je einen Test für

1. Gegeben `char c = '1'` → Umwandlung in `int`
2. Gegeben `int i = 127` → Umwandlung in `byte`

und prüfe das Ergebnis, also den erwarteten Wert, z.B. mithilfe von `System.out.println` oder mit der "Assertions-Methode" `assertEquals(<expected>, <actual>)`.